

ANALYSIS AND DESIGN OF ALGORITHMS

Instructor: Dr. Mert Büyükdede



Greedy Method

The **Greedy Method** is an algorithmic approach that builds up a solution piece by piece, always choosing the option that looks best at the current step.

It aims to find a **global optimum** by making a series of **locally optimal choices**.

Local optimum: Best choice at a particular step.

Global optimum: Best possible overall solution.

In short: A greedy algorithm makes the **best possible choice at each step**, without thinking about the future consequences.



Key Properties of Greedy Method

1. Greedy Choice Property

A global optimum can be arrived at by choosing a local optimum at each step.

2. Optimal Substructure

A problem exhibits optimal substructure if an optimal solution to the problem contains optimal solutions to its subproblems.

If a problem satisfies both properties, the greedy method will likely produce an optimal solution.



General Steps of a Greedy Algorithm

1. **Analyze the problem** carefully.
2. **Identify the selection criteria** to determine the best local choice.
3. **Make the locally optimal choice** based on the criteria.
4. **Update the problem** according to the chosen solution.
5. **Repeat** the process until the entire problem is solved.



Advantages and Disadvantages of Greedy Method

Advantages	Explanation
Simple and Easy to Implement	Greedy algorithms are intuitive and require fewer lines of code compared to dynamic programming or other complex methods.
Fast Execution (Efficient Time Complexity)	Often runs in $O(n \log n)$ or $O(n)$ time, making it suitable for large inputs where quick solutions are needed.
Low Memory Usage (Space-Efficient)	Requires minimal memory, as it doesn't store all possible states or solutions — works with the current choice only.
Good for Many Practical Applications	Used successfully in networking, scheduling, and resource optimization tasks where fast approximate solutions are acceptable.
Suitable for Approximation Algorithms	Even when optimal solutions are not guaranteed, greedy algorithms often produce sufficiently good approximations quickly.

Advantages and Disadvantages of Greedy Method

Disadvantages	Explanation
May Not Find the Global Optimum	Greedy choices optimize locally but can lead to poor overall solutions if the problem structure is not suitable.
Applicable Only to Certain Problems	Works only if the problem satisfies Greedy Choice Property and Optimal Substructure; otherwise, greedy methods fail.
Myopic (Short-Sighted) Behavior	Only considers immediate best choices without analyzing long-term consequences or future possibilities.
No Backtracking or Reconsideration	Once a choice is made, it cannot be undone; greedy methods do not revisit previous steps even if better options arise later.
Fails for Complex Structured Problems	For problems with intricate interdependencies (e.g., 0/1 Knapsack), greedy can easily miss the optimal combination.

Examples of Problems Solved by Greedy Method

Coin Change Problem

Given coins of certain denominations, find the minimum number of coins needed to make a specific amount of money.

Example: To make 63 units, prefer using the largest coins first (50, 10, 1).

Aim: To make this units using the least amount of coins

Note: Greedy approach works with some coin systems (like Turkey's), but may fail with others.



Example

Problem:

You have coins of 1, 5, 10, and 25 units.

Find the minimum number of coins needed to make 63 units.

Solution (Greedy Approach):

1. Pick the largest coin.
2. $63 \rightarrow 25 \rightarrow$ Remaining: 38
3. $38 \rightarrow 25 \rightarrow$ Remaining: 13
4. $13 \rightarrow 10 \rightarrow$ Remaining: 3
5. $3 \rightarrow 1 \rightarrow$ Remaining: 2
6. $2 \rightarrow 1 \rightarrow$ Remaining: 1
7. $1 \rightarrow 1 \rightarrow$ Remaining: 0

Coins used: $2 \times 25, 1 \times 10, 3 \times 1$

 **Answer: 6 coins**

Example

Given Coins: ₹1, ₹3, ₹4

Target Amount: ₹6

Greedy Approach:

1. Pick the largest coin $\leq$ ₹6 $\rightarrow$ ₹4

Remaining amount: ₹6 - ₹4 = ₹2

2. Pick the largest coin $\leq$ ₹2 $\rightarrow$ ₹1

Remaining amount: ₹2 - ₹1 = ₹1

3. Pick the largest coin $\leq$ ₹1 $\rightarrow$ ₹1

Remaining amount: ₹1 - ₹1 = ₹0

Coins used by Greedy: ₹4, ₹1, ₹1

Total coins used: 3 coins

Optimal Solution:

• Pick two ₹3 coins: ₹3 + ₹3 = ₹6

Coins used: ₹3, ₹3

Total coins used: 2 coins

Conclusion:

• **Greedy method result:** 3 coins

• **Optimal result:** 2 coins

● Thus, the Greedy method fails to find the optimal solution with the fewest coins.

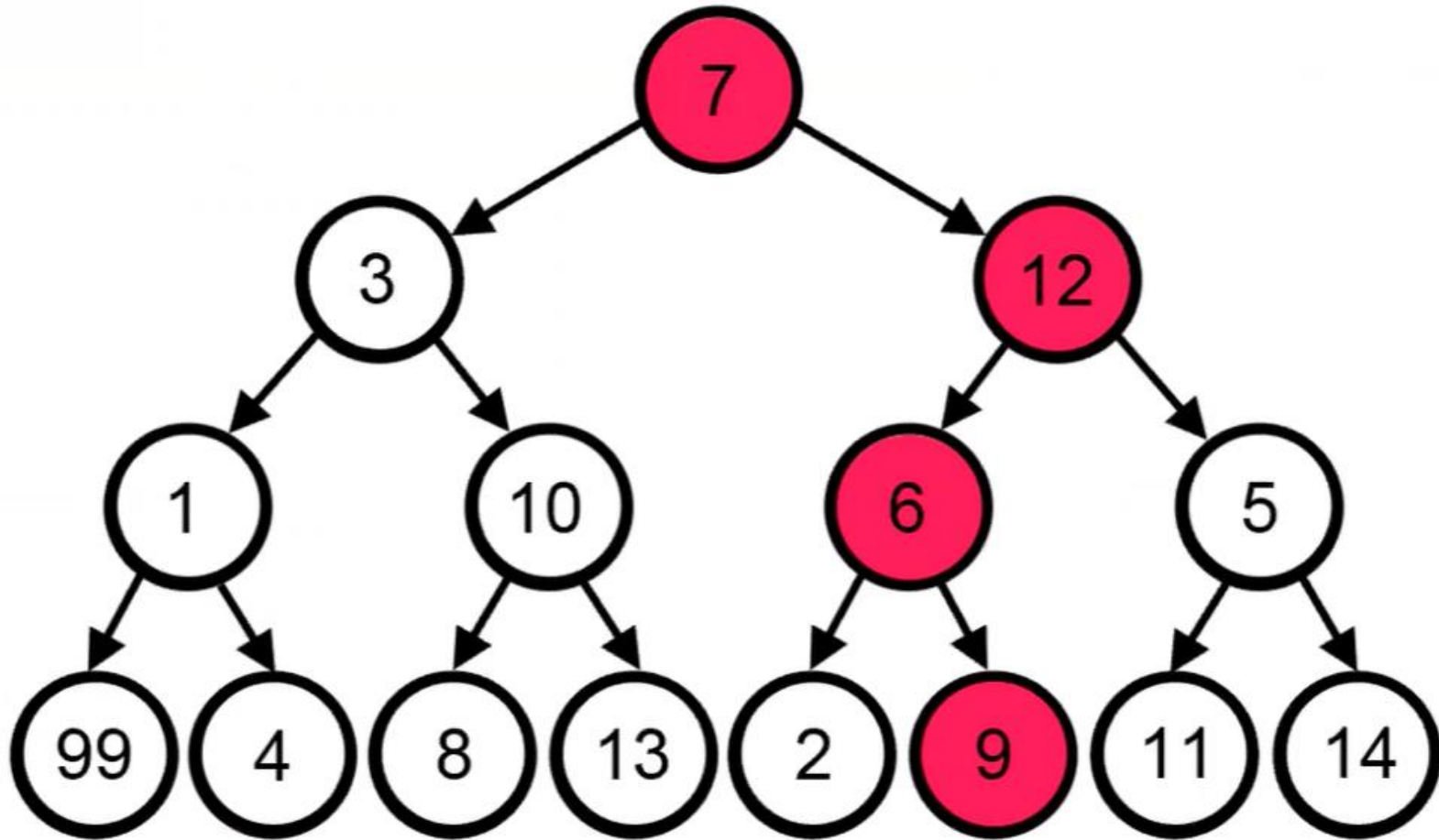
Examples of Problems Solved by Greedy Method

If greedy always gives the correct solution for a monetary system, it is called a "canonical coin system".

For example, the American coin (1, 5, 10, 25) is canonical. But the (1, 3, 4) system is not canonical.



Which option is best for you?



Activity: "Real-Life Coin Change Problem"

Problem Scenario:

A customer made a purchase of 135 TL in a café and paid 200 TL. The cashier is required to give back the change of 65 TL using the fewest number of coins. The available denominations of coins are as follows:

- 1 TL
- 5 TL
- 10 TL
- 25 TL
- 50 TL

Goal 1:

Let's try to create the pseudo code for this problem

Activity: "Real-Life Coin Change Problem"

Problem Scenario:

```
Function CoinChange(coins[], change):  
    Sort coins[] in descending order  
    Initialize coinCount = 0  
    For each coin in coins:  
        While change >= coin:  
            change = change - coin  
            coinCount = coinCount + 1  
    Return coinCount
```


Activity: "Real-Life Coin Change Problem"
Problem Scenario:

Goal 2:

**Let's try to create the java code for this problem and
try different paid coins**



```
import java.util.Arrays;
import java.util.Collections;

public class CafeChange {

    public static int coinChange(int[] coins, int change) {
        // Sort coins in descending order
        Integer[] coinsArray = Arrays.stream(coins).boxed().toArray(Integer[]::new);
        Arrays.sort(coinsArray, Collections.reverseOrder());

        int coinCount = 0;

        // Loop through each coin
        for (int coin : coinsArray) {
            while (change >= coin) {
                change -= coin; // Decrease change by coin value
                coinCount++; // Increment coin count
            }
        }
    }
}
```

```
        return coinCount; // Return the total number of coins used
    }

    public static void main(String[] args) {
        int[] coins = {1, 5, 10, 25, 50}; // Example coins available
        int change = 65; // Change to be returned

        int result = coinChange(coins, change);
        System.out.println("Minimum coins needed: " + result); // Output the result
    }
}
```

Activity: "Real-Life Activity Selection Problem"

Problem Scenario:

You are a conference organizer planning a day of multiple sessions. The goal is to select as many non-overlapping sessions as possible for attendees to attend, given that each session has a start time and an end time. There are constraints on the maximum number of rooms available, and each room can only host one session at a time.

You are given the following list of sessions:

Session	Start Time	End Time
S1	09:00	10:30
S2	09:30	11:00
S3	10:30	12:00
S4	11:00	12:30
S5	13:00	14:00
S6	12:30	14:00
S7	14:30	16:00
S8	15:00	16:30
S9	16:00	17:00

Goal I:

Let's try to create the pseudo code for this problem

```
Function ActivitySelection(sessions):  
    Sort sessions by end time  
    Initialize lastEndTime = 0  
    Initialize selectedActivities = []  
  
    For each session in sessions:  
        If session.startTime >= lastEndTime:  
            Add session to selectedActivities  
            lastEndTime = session.endTime  
  
    Return selectedActivities
```


Goal 2:

Let's try to create the java code for this problem.

```
import java.util.*;

public class ActivitySelection {

    // Activity class to store start and end times
    static class Activity {
        int startTime;
        int endTime;

        Activity(int startTime, int endTime) {
            this.startTime = startTime;
            this.endTime = endTime;
        }
    }
}
```

```
public static void activitySelection(List<Activity> sessions) {  
    // Sort activities by their end time  
    sessions.sort(Comparator.comparingInt(a -> a.endTime));  
  
    int lastEndTime = 0;  
    List<Activity> selectedActivities = new ArrayList<>();  
  
    for (Activity session : sessions) {  
        // If the session starts after the last selected session ends  
        if (session.startTime >= lastEndTime) {  
            selectedActivities.add(session);  
            lastEndTime = session.endTime;  
        }  
    }  
  
    // Display selected activities  
    System.out.println("Selected sessions: ");  
    for (Activity activity : selectedActivities) {  
        System.out.println("Session: " + activity.startTime + " - " + activity.endTime);  
    }  
}
```

```
public static void main(String[] args) {  
    // Create a list of sessions  
    List<Activity> sessions = new ArrayList<>();  
    sessions.add(new Activity(9, 10));  
    sessions.add(new Activity(9, 11));  
    sessions.add(new Activity(10, 12));  
    sessions.add(new Activity(11, 12));  
    sessions.add(new Activity(13, 14));  
    sessions.add(new Activity(12, 14));  
    sessions.add(new Activity(14, 16));  
    sessions.add(new Activity(15, 16));  
    sessions.add(new Activity(16, 17));  
  
    // Call the method to select activities  
    activitySelection(sessions);  
}  
}
```

What is the Knapsack Problem?

The **Knapsack Problem** is a classical **optimization problem** in computer science and operations research.

The scenario:

You have a **backpack (knapsack)** with limited capacity, and you want to fill it with items to **maximize the total value, without exceeding its weight limit**.

Problem Definition

You are given:

- n items
- Each item has:
 - A **weight** w
 - A **value** v
- A knapsack with maximum capacity W

Goal:

Choose a set of items such that the **total weight** $\leq W$, and the **total value** is as high as possible.

✌️ Types of Knapsack Problems

1. 0–1 Knapsack

- Each item can **either be taken as a whole or not taken at all**.
- You make a binary decision: include (1) or exclude (0).

2. Fractional Knapsack

- Items can be **broken into smaller pieces**.
- You are allowed to take a **fraction** of an item.
- This version can be solved **using a Greedy algorithm**.

3. Unbounded Knapsack

- You can take **unlimited copies** of each item.
- Useful for problems like coin change or supply stock management.

✓ Fractional Knapsack: Greedy Approach

This version allows you to **take parts of items**.

🎯 Idea:

Take the items **with the highest value-to-weight ratio first**, until the bag is full.

1 2 3 4 Steps:

1. Calculate value/weight ratio for all items.
2. Sort items by this ratio in **descending** order.
3. Add as much as possible from each item to the knapsack.

Time Complexity:

- $O(n \log n)$ (due to sorting)

Example

You are given:

Item	Weight	Value
1	2	12
2	1	10
3	3	20
4	2	15

Knapsack capacity $W=5$

For 0–1 Knapsack:

- Best combination: item 1 + item 2 + item 4 $\rightarrow$ weight = $2 + 1 + 2 = 5$
- Value = $12 + 10 + 15 = \mathbf{37}$

For Fractional Knapsack:

- Take item 2 (whole) $\rightarrow$ 1kg, 10 value
- Take item 4 (whole) $\rightarrow$ 2kg, 15 value
- Take 50% of item 3 $\rightarrow$ 1.5kg, 10 value
- Total weight = 4.5kg
- Total value = $10 + 15 + 10 = \mathbf{35}$

Real-Life Applications

- Cargo or shipping optimization
- Budget allocation problems
- Memory or resource management
- Travel packing
- Stock and investment selection
- Advertisement Selection in Online Platforms
- Diet or Nutrition Planning
- Cloud Computing – Virtual Machine (VM) Placement
- Product Assortment in Retail Stores



Summer Shape-Up – Optimize Your Meal Plan!

The Story:

Summer is around the corner, and many university students are motivated to get in shape, feel healthier, and look their best. As part of a **campus health challenge**, students are encouraged to follow a **high-protein, calorie-controlled meal plan** for better energy, muscle maintenance, and weight control.

You are a **software/computer engineering student** collaborating with a team of **nutritionists** to develop a **smart diet planning tool**. Your job is to build the **algorithm behind the tool**, which will automatically generate the most protein-rich daily meal plan **without exceeding a calorie limit**.

Goal: Maximize protein

Constraint: Do not exceed 1800 kcal

Data: 12 food items with calorie and protein values

ID	Food Item	Calories per Serving	Protein per Serving (grams)
1	Grilled Chicken Breast	165 kcal	31 g
2	Boiled Egg	78 kcal	6 g
3	Brown Rice (1 cup)	216 kcal	5 g
4	Yogurt (plain)	100 kcal	10 g
5	Almonds (10 pcs)	70 kcal	2.5 g
6	Protein Shake	130 kcal	24 g
7	Avocado (half)	120 kcal	2 g
8	Broccoli (1 cup)	55 kcal	3.7 g
9	Tuna (canned, in water)	194 kcal	29 g
10	Apple (medium)	95 kcal	0.5 g
11	Lentil Soup (1 bowl)	180 kcal	12 g
12	Cottage Cheese	98 kcal	11 g

HOMEWORK : Pseudocode Writing Write a Greedy algorithm that selects food items based on the best protein-per-calorie ratio, allowing fractional servings.



Homework : Java Implementation

Let's write the Java code according to the algorithm you wrote.